

单元四：数据结构初步

第七章 数组

内容取自：《C Primer Plus（第五版）中文版

•6.11 数组

•10.1 数组



7.1 总结与回顾（数据类型与程序设计）

7.2 数组--数据的存储结构

7.3 数组的声明、操作和使用

7.4 字符数组（字符串及操作）

7.5 数组作为函数参数的处理

7.6 数组的排序、查找算法



1. 高级语言的基本能力
 - 简单数据类型（整型/实型/字符型）
 - 程序控制结构（顺序/分支/循环、子程序调用）
2. 算法与算法设计的基本方法
 - 算法是求解问题的基本思路和步骤。
 - 采用自顶向下/逐步求精的算法设计方法。
3. 数据类型+算法=程序

4. 数据类型的内涵（三要素）

- 数据的抽象（定义）
 - 整数/实数/字符等是对客观世界事物（数据）的抽象，逻辑表达。
- 数据的存储空间（范围）
 - 长整型、短整形、浮点数、字符等数据所分配的存储空间各不相同。
- 数据的操作
 - 算术、逻辑、关系运算。

5. 求解问题的局限：

— 数据抽象的局限

- 很多客观世界的事物很难抽象到简单的数据类型上。比如：数列或级数的抽象。

— 存储能力的局限（狗熊掰棒子）

— 处理能力的局限

- 比如对任意数列的排序和查找问题，依照目前的能力尚不能够进行处理。

• 解决上述问题的基本思路：

增强数据的表达、存储和处理能力。

7.1 总结与回顾（数据类型与程序设计）

7.2 数组--数据的存储结构

7.3 数组的声明、操作和使用

7.4 字符数组（字符串及操作）

7.5 数组作为函数参数的处理

7.6 数组的排序、查找算法



7.2 数组--数据存储结构

- 引入数组是为了增强对复杂数据的**存储能力、操作能力**。比如某个班学生的成绩、候选人的计票、每天的温度等。
- 数组是一种数据的存储结构，因此需要了解：
 - 对表达对象（数据的抽象）的支持
 - 数组的存储结构
 - 数组的操作

7.2 数组--数据存储结构

1. 数组对表达对象的支持

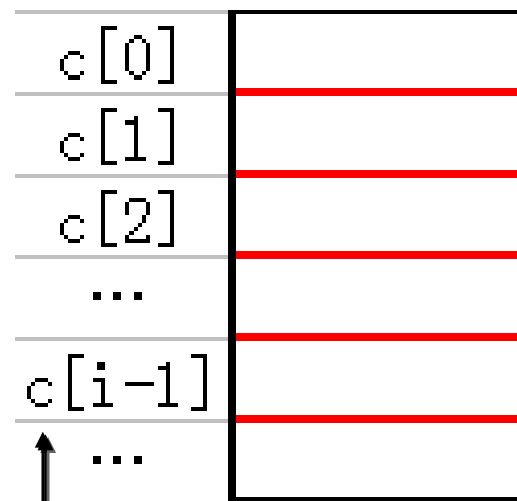
- 现实世界经常存在这样的数据：某课程一个班级中按学号排列的各个学生考试成绩、一年中各月份的平均温度……其特点是这有限个数据元素的类型相同，且具有先后顺序关系；
- 上述特征的数据可以抽象为**线性表、向量**。
- **线性表（向量）**是具有相同数据类型的 $n(n \geq 0)$ 个数据元素的有限序列，通常记为：
 $(a_1, a_2, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$
其中 n 为表长， $n=0$ 时称为空表。
线性表数据元素之间为线性关系，通俗讲就是“一个接一个的排列”。
- **数组是对线性表的一种存储结构。**

7.2 数组--数据存储结构

2. 数组是一组**连续**的存储单元（存储结构）

- **连续**的含义：这些存储单元位置相邻（一个接一个），可以容纳多个具有相同数据类型的数据元素(数据项)，这些数据元素具有相同的名字

- 存储位置的相邻性体现了线性表数据元素之间的相邻性。

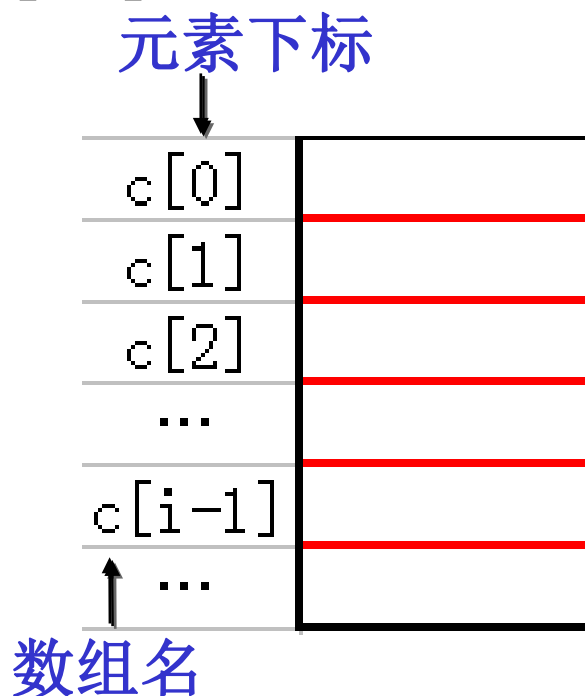


数组名（数组中的所有元素具有相同的名字c）

7.2 数组--数据存储结构

3. 数组元素的访问（操作）

- **数组名[元素序号]**。其中元素序号又称下标。
- 注意：第一个元素的序号为0，因此 $c[0]$ 引用数组 c 的第一个元素。 $c[i-1]$ 引用第 i 个元素。



7.2 数组--数据存储结构

(3.续) 下标运算符 “[]” (数组操作)

- 在赋值语句**右边**时，该操作从数组元素中检索数据；

例： $x=c[1]$ ； //读取下标为1的数组元素的值，赋给变量x；

- 在左边时，指对数组元素的赋值。

例： $c[2]=x*3+5$ ；//将赋值表达式右边的值赋给下标为2的数组元素

c[0]	
c[1]	
c[2]	
...	
c[i-1]	
...	

切记：数组是一种存储结构。

7.1 总结与回顾（数据类型与程序设计）

7.2 数组--数据的存储结构

7.3 数组的声明、操作和使用

7.4 字符数组（字符串及操作）

7.5 数组作为函数参数的处理

7.6 数组的排序、查找算法



7.3 数组的声明、操作和使用

7.3.1 数组的声明

7.3.2 数组的初始化

7.3.3 数组逐元素操作

C语言中如何使用数组呢?



7.3.1 数组的声明

一、数组的声明：（C99标准之前）

语法格式：元素类型名 数组名[整型常量表达式];

元素类型名 可以是整型、字符型、浮点型、结构和指针。

常量表达式说明数组元素的个数，运算结果必须为整型。

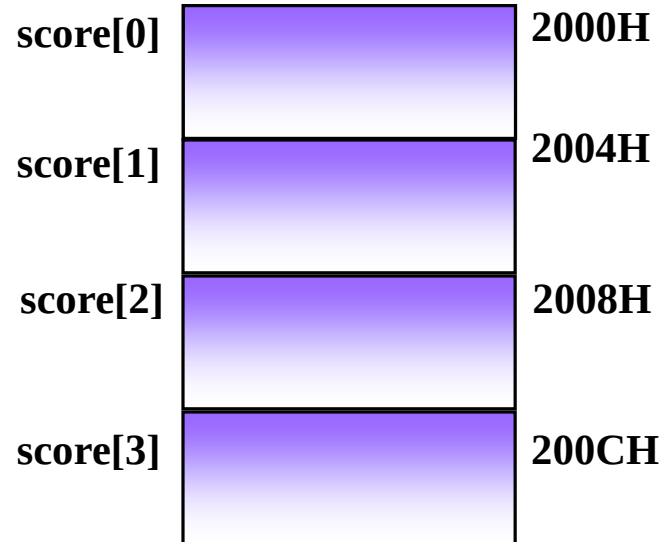
存储空间

例如：

```
int main()
{
    int score[4];
    .....
}
```

或者

```
#define SIZE 4
int main()
{
    int score[SIZE];
    .....
}
```



7.3.1 数组的声明

一、数组的声明：（C99标准）

允许声明变长数组

元素类型名 数组名[整型表达式];

```
int n;
```

```
scanf("%d",&n);
```

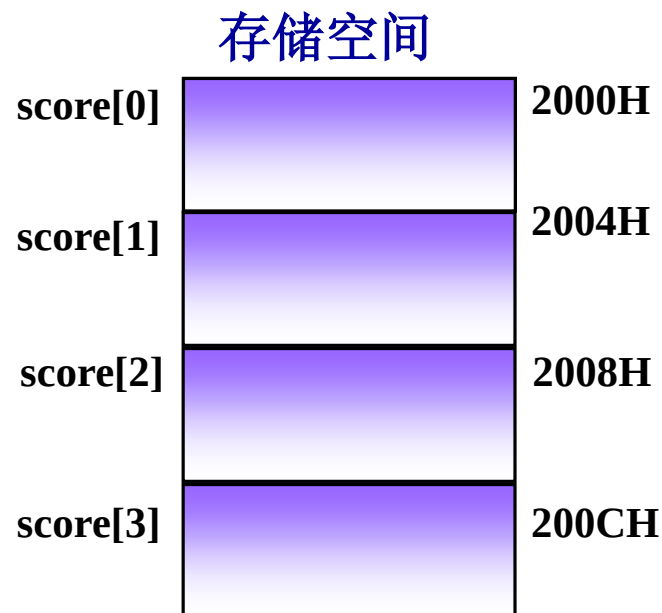
```
int score[n];//数组长度为变量
```

7.3.1 数组的声明

1. 数组中各元素的类型相同;
2. C语言中数组下标**从0开始**, `score[0]`表示第1个数组元素, `score[i]`表示第*i*-1个数组元素(*i*是变量)。
3. 允许在同一个类型说明中, 说明多个数组和多个变量。例如:

```
int num, students[150], score[4];
```

但注意数组名不能和其他变量名相同。



•数组元素的引用:

数组名[下标], 如`score[0]`;

•下标必须是整数或者整数表达式。注意: 表达式中的操作数可以是常量、变量、函数调用或表达式。

7.3.2 数组的初始化

二. 数组的初始化

C语言中可利用声明语句对数组元素的值进行初始化:

元素类型名 数组名[整型表达式]={值, 值……值};
存储空间

例如:

```
int score[4]={65, 78, 54, 91};
```

数组初始化是在编译阶段进行的。这样将减少运行时间，提高效率。

score[0]	65	2000H 2001H
score[1]	78	2002H 2003H
score[2]	54	2004H 2005H
score[3]	91	2006H 2007H

7.3.2 数组的初始化

1. 若在定义一个整型数组的同时赋初值，如果初始化值的个数小于数组元素的个数，剩余的元素被自动初始化为0。

例如： `int score[4]={80};`

存储空间

注意：整型数组各元素不会自动初始化为0，至少要把第一个数组元素初始化为0，才能使剩下的元素自动初始化为0。

score[0]	80	2000H 2001H
score[1]	0	2002H 2003H
score[2]	0	2004H 2005H
score[3]	0	2006H 2007H

7.3.2 数组的初始化

2. 若在定义一个浮点型数组的同时赋初值，如果初始化值的个数小于数组元素的个数，剩余的元素被自动初始化为0.000000。

如：`float score[4]={85.5};`

关于字符型数组后面再讲。

7.3.2 数组的初始化

3. 如果初始化元素个数大于数组长度，则编译会报错，例如：

```
int score[4]={65, 78, 54, 91,60};
```

4. 如果在声明带有初始化值列表的数组时省略数组的大小，那么数组元素的个数就是初始化值列表中的元素个数。

```
int score[]={65, 78, 54, 91,60};//score有5个元素
```

5. C99新特性：在初始化列表中使用带有方括号的元素下标可以初始化某个特定的元素

```
int score[]={[4]=60};//把score[4]初始化为60
```

```
int days[MONTHS]={31,28,[4]=31,30,31,[1]=29};
```

```
//days元素依次为： 31,29,0,0,31,30,31,0,0,0,0,0
```

7.3.3 数组的逐元素操作

- 数组元素和普通的基本类型变量一样，对基本类型变量的所有操作(读、赋值、取地址等)同样适用于数组元素，基本变量能出现的地方数组元素也可以出现。

- **数组元素的引用：**

数组名[下标]，如score[0];

- **下标必须是整数或者整数表达式。注意：表达式中的操作数可以是常量、变量、函数调用或表达式。**

7.3.3 数组的逐元素操作

- C语言规定数组元素不能整体引用，每次只能引用数组的一个元素。例如，不能用赋值运算对数组进行整体赋值。因为在C语言中，**数组名具有特殊含义，它代表数组的首地址。**

```
int a[5];
```

```
a = {1,2,3,4,5};//错误
```

数组名实际上是数组的首元素地址

```
int main()
{
    char array[5];
    printf("array = %p &array[0] = %p", array , &array[0]);
    return 0;
}
```

array = FFF0

&array[0] = FFF0

7.3.3 数组的逐元素操作

三、数组的逐元素操作：

一般情况下(字符数组会有例外)，无法对数组进行整体操作，例如整体赋值或者输出。要采用循环控制结构对数组的元素逐个进行操作和处理。

例1：读入10个分数然后进行处理

请同学们体会下标运算符的使用，以及数组元素的值和数组元素下标的区别。

7.3.3 数组的逐元素操作

```
#include <stdio.h>
#define SIZE 10
#define PAR 72
int main(void)
{
    int index, score[SIZE];
    int sum = 0;
    float average;

    printf("Enter %d golf scores:\n", SIZE);
    for (index = 0; index < SIZE; index++)
        scanf("%d", &score[index]); // read in the ten scores
```

定义常量SIZE，用于表示数组长度，便于程序的阅读和修改。

注意书写习惯：常量名全部大写，变量名小写

7.3.3 数组的逐元素操作

```
printf("The scores read in are as follows:\n");
for (index = 0; index < SIZE; index++)
    printf("%5d", score[index]); // verify input
printf("\n");

for (index = 0; index < SIZE; index++)
    sum += score[index];        // add them up
average = (float) sum / SIZE;   // time-honored method

printf("Sum of scores = %d, average = %.2f\n", sum, average);
printf("That's a handicap of %.0f.\n", average - PAR);

return 0;
}
```

7.3.3 数组的逐元素操作

- 注意**越界控制**：不要引用超出数组范围的数组元素。出于执行速度的考虑，系统运行时不会自动检测元素下标是否越界，因此编写程序时要格外小心，由编程人员自己确保对元素的正确引用，以免因下标越界对其他存储单元中数据造成破坏。

存储空间

如：

```
int score[4]={65, 78, 54, 91};  
printf(“%d”, score[4]); /*编译  
不会报错，但是输出结果是未  
知的*/
```

score[0]	65	2000H
		2001H
score[1]	78	2002H
		2003H
score[2]	54	2004H
		2005H
score[3]	91	2006H
		2007H
	?	2008H
		2009H28

7.3.3 数组的逐元素操作

练习2. 定义一个能容纳10个元素的整形数组a，从键盘读取9个整数存放到前9个数组元素中。

- 1) 从键盘读取一个整数n和位置p($0 \leq p \leq 8$)，插入n到数组a中，插入位置：下标p。要求插入点及后续的数组元素都要后移动。
- 2) 然后，从键盘再次读取位置p($0 \leq p \leq 8$)，删除数组a中下标为p的元素。要求被删除的数组元素之后的元素都要前移。

7.3.3 数组的逐元素操作

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
10	20	30	45		

插入元素

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
10	20		30	45	

a[0]	a[1]	a[2]	a[3]	a[4]
10	20	30		
a[0]	a[1]	a[2]	a[3]	a[4]
50	30			

删除元素

7.3.3 数组的逐元素操作

//插入num到下标为p的数组元素中

for(i=SIZE-2;i>=p;i--)//下标为SIZE-2~p的数组元素依次后挪

a[i+1]=a[i];

a[p]=num;//插入num到下标为p的数组元素中

//删除下标为p的数组元素中内容

for(i=p+1;i<=SIZE-1;i++)//下标为p+1~SIZE-1的数组元素依次前挪

a[i-1]=a[i];

7.3.3 数组的逐元素操作

```
int main(void)
{
    int a[SIZE],i,p,num;

    //往数组中前SIZE-1个元素赋值
    printf("input %d numbers\n",SIZE-1);
    for(i=0;i<=SIZE-2;i++)
        scanf("%d",&a[i]);

    //输出数组
    printf("before insert,the array is:\n");
    for(i=0;i<=SIZE-2;i++)
        printf("%d\t",a[i]);
```


7.3.3 数组的逐元素操作

//读入要插入的元素以及位置下标

```
printf("input number to add:\n");  
scanf("%d",&num);  
printf("input position(<=%d):\n",SIZE-2);  
scanf("%d",&p);
```

//插入 元素

```
for(i=SIZE-2;i>=p;i--)//下标为SIZE-2~p的数组元素依次后挪  
    a[i+1]=a[i];  
a[p]=num;//插入num到下标为p的数组元素中
```

7.3.3 数组的逐元素操作

//读入要删除的元素下标

```
printf("\ninput number position to delete:\n");
```

```
scanf("%d",&p);
```

```
for(i=p+1;i<=SIZE-1;i++)//下标为p+1~SIZE-1的数组元素依次前挪
```

```
    a[i-1]=a[i];
```

```
printf("after delete,the array is:\n");
```

```
for(i=0;i<=SIZE-2;i++)
```

```
    printf("%d\t",a[i]);
```

```
return 0;
```

```
}
```

7.3.3 数组的逐元素操作

例3. 40位学生为餐厅打分，分数分为1~10的10个等级，要求统计出各个分值的打分人数；

- 分析：
 - 40个学生的打分分值构成了一个线性表，可以用一个长为40的整型数组来存放；

int response[40]

- 假设 a_i 代表分值为 i 的票数，则 $(a_1, a_2, \dots, a_{10})$ 构成一个线性表，可以用一个长度为10的整型数组来存放。**int frequency[10]**

7.3.3 数组的逐元素操作

```
#include <stdio.h>
#define RESPONSE_SIZE 40
#define FREQUENCY_SIZE 11
int main(void)
{
    int response[RESPONSE_SIZE]; //存放学生评分
    int frequency[FREQUENCY_SIZE]={0};/*存放1~10之间各
                                         种等级分值的统计票数*/
    int answer,rating;

    for(answer=0;answer<RESPONSE_SIZE;answer++)
        scanf("%d",&response[answer]);//读取打分
```

7.3.3 数组的逐元素操作

```
for(answer=0;answer<RESPONSE_SIZE;answer++)  
    ++frequency[response[answer]];//统计  
  
for(rating=1;rating<FREQUENCY_SIZE;rating++)  
    printf(" %d-%d\n",rating,frequency[rating]);//输出结果  
  
return 0;  
}
```

7.3.3 数组的逐元素操作

1. `int response[RESPONSE_SIZE]` 存放40个学生的评分
`int frequency[FREQUENCY_SIZE]` 存放1~10之间各种等级分值的统计票数
2. `response[answer]` 某个学生的评分值，取值为1~10
3. `frequency[rating]` 某一分值的统计结果
4. `++frequency[response[answer]]` 根据某个学生的评分值将该分值的统计结果加1。
5. 为什么要将FREQUENCY_SIZE定义为11呢？
“将数组的下标值与评分值对应”。使用
`frequency[1]~frequency[10]`

7.1 总结与回顾（数据类型与程序设计）

7.2 数组--数据的存储结构

7.3 数组的声明、操作和使用

7.4 字符数组（字符串及操作）

7.5 数组作为函数参数的处理

7.6 数组的排序、查找算法



四、字符数组（字符串）

- 假设有一个字符串 “first ”，能否用一个变量存放？
- 在C语言中没有专门的字符串变量。通常用一个字符数组来存放一个字符串，并以字符 ‘\0’ 作为字符串的结束符。 ‘\0’ 对应的ASCII码为0。

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
f	i	r	s	t	\0

- 在声明一个容纳字符串的字符数组时，数组的大小应足以容纳字符串中的字符以及字符串结束符 ‘\0’ 。

➤ 字符数组初始化

三种方式：

`char a[]={'f', 'i', 'r', 's', 't', '\0'};`

也可写为：`char a[]={"first"};`

或去掉{}写为：`char a[]="first";`

内存中存放形式为：

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
f	i	r	s	t	\0

后两种初始化情况中，`'\0'` 是系统自动加上的。

上述声明中没有指定数组的长度，编译器会根据字符串的长度来确定数组的长度（字符串长度+1）。

注意：对一个字符数组，如果不作初始化赋值，则必须**说明数组长度,否则编译出错！**

在采用字符串方式后，字符数组的输入输出有以下几种方式：

- 1.定义字符数组时进行初始化。**
- 2.使用循环语句逐个地输入、输出每个字符。**
- 3.用printf函数/scanf函数、gets函数/puts函数一次性输出、输入一个字符数组中的所有字符。**

用printf函数和scanf函数一次性输出、输入一个字符数组中的所有字符

```
int main()
{
    char st[200];
    printf("input string: \n");
    scanf("%s", st); /*注意:是st而不是&st!因为数组名代表了数组的首地址*/
    printf("%s", st); /*注意:转换说明符是%s*/
}
```

scanf函数读取用户键入的字符到字符数组，直到遇到空格、回车、或文件结束符（EOF）为止。空格、回车、或文件结束符被丢弃，最后一个字符读入后往字符数组中写入结束符‘\0’。printf输出时‘\0’不输出。

注意：scanf不关心字符数组的大小，所以它往数组中写字符时，所写入的字符可能超出数组的范围。程序员必须自己控制。

7.4 字符数组

遇空格结束读取

input string:

aaaaa aa ✓

aaaaa请按任意键继续...

'a'	'a'	'a'	'a'	'a'	'\0'	
-----	-----	-----	-----	-----	------	-------

遇回车结束读取

input string:

aa

aaaaaaaaaaaaa ✓

aa

aaaaaaaaaaaaa请按任意键继续...

```
scanf(“%s”, &st[1]); /*将输入存放到字符数  
组第二个单元起的连续单元中*/  
printf(“%s”, &st[1]); /*输出字符数组中从第二  
个字符开始的所有字符。注意必须有&, 否则出  
错!*/
```

读、写一行字符：

- **char * gets(char * s);**

从标准输入设备（如键盘）读取字符到s所指向的数组中，直到读到文件末尾或者新行符‘\n’。新行符被丢弃，最后一个字符读入后写入一个‘\0’。若成功则返回s，若无字符读入数组或者读取失败返回空指针NULL。

- **int puts(const char * s);**

将s所指向的字符串输出到标准输出设备（如显示器），并在输出中添加一个新行符‘\n’。终止字符串的‘\0’不被输出。

7.4 字符数组

```
int main()
{
    char st[15];
    printf("input string:\n");
    gets(st); //读一行字符
    puts(st); //写一行字符

    return 0;
}
```

input string:
hello /
hello
请按任意键继续...

'h'	'e'	'l'	'l'	'o'	'\0'	
-----	-----	-----	-----	-----	------	-------

scanf和gets在读取一行数据时的区别:

```
int main()
```

```
{
```

```
    char str[10];
```

```
    gets(str);
```

```
    if (str[0]=='\0')
```

```
        printf("is null");
```

```
    return 0;
```

```
}
```

↙
is null请按任意键继续...

使用gets，若只是输入回车，则未读取任何字符到数组。

'\0'						
------	--	--	--	--	--	-------

7.4 字符数组

```
int main()
{
    char str[10];
    scanf("%s",str);
    if (str[0]=='\n')
        printf("is enter");
    else if (str[0]=='\0')
        printf("is null");
    return 0;
}
```



使用scanf，若只是输入回车或者空格，则程序一直在等待，直到输入一个非回车、非空格字符。

即：scanf无法实现空行的读取！

• 用puts函数输出的字符串中可以包含转义字符。例如：

• `char str [] = { " China\rBeijing" } ;`

• `puts(str);`

• 输出结果：

• China

• Beijing

• 在输出时，将字符串

• 结束标志 ' \0 '

• 转换成 ' \n ' ，

• 即输出完字符串后换行。

• 3. strcat函数

- 其一般形式为: **strcat(字符数组1, 字符数组2)**
- Strcat的作用是连接两个字符数组中的字符串, 把字符串2接到字符串1的后面, 结果放在字符数组1中, 函数调用后得到一个函数值——字符数组1的地址。



- `char str1 [30] = { " People' s Republic of " } ;`

- `char str2 [] = { " China" } ;`

- `printf(" %s" , strcat(str1, str2)) ;`

- 输出:

- People' s Republic of China

str1:	P	e	o	p	l	e	'	s	␣	R	e	p	u	b	l	i	c	␣	o	f	␣	\0	\0	\0	\0	\0	\0	\0	\0	\0
str2:	C	h	i	n	a	\0																								
str3:	P	e	o	p	l	e	'	s	␣	R	e	p	u	b	l	i	c	␣	o	f	␣	C	h	i	n	a	\0	\0	\0	\0

• 4. strcpy函数

- 其一般形式为: `strcpy(字符数组1, 字符串2)`
- `strcpy`是“字符串复制函数”。作用是将字符串2复制到字符数组1中去。例如:
- `char str1 [10] , str2 [] = { " China" } ;`
- `strcpy(str1, str2) :`

C	h	i	n	a	\0	\0	\0	\0	\0
---	---	---	---	---	----	----	----	----	----

关于strcpy函数的几点说明

- (1) 字符数组1必须定义得足够大，以便容纳被复制的字符串。字符数组1的长度不应小于字符串2的长度。
- (2) “字符数组1”必须写成数组名形式(如str1)，“字符串2”可以是字符数组名，也可以是一个字符串常量。如strcpy(str1, "China");

- (3) 复制时连同字符串后面的 ' \0 ' 一起复制到字符数组1中。
- (4) 可以用 `strncpy` 函数将字符串2中前面若干个字符复制到字符数组1中去。例如: `strncpy(str1, str2, 2);`
- 作用是将 `str2` 中前面2个字符复制到 `str1` 中去, 然后再加一个 ' \0 '。



• (5) 不能用赋值语句将一个字符串常量或字符数组直接给一个字符数组。如：

- `str1=" China" ;` 不合法
- `str1=str2;` 不合法
- 用strcpy函数只能将一个字符串复制到另一个字符数组中去。
- 用赋值语句只能将一个字符赋给一个字符型变量或字符数组元素。下面是合法的使用：
 - `char a [5] , c1, c2;`
 - `c1=' A' ; c2=' B' ;`
 - `a [0] =' C' ; a [1] =' h' ; a [2] =' i' ;`
 - `a [3] =' n' ; a [4] =' a' ;`

•5. strcmp函数

- 其一般形式为： `strcmp(字符串1, 字符串2)`
- `strcmp`的作用是比较字符串1和字符串2。
- 例如：** `strcmp(str1, str2);`
- `strcmp(" China" , " Korea" );`
- `strcmp(str1, " Beijing" );`

• 比较的结果由函数值带回

- (1) 如果字符串1=字符串2, 函数值为0。
- (2) 如果字符串1>字符串2, 函数值为一正整数。
- (3) 如果字符串1<字符串2, 函数值为一负整数。
- **注意:** 对两个字符串比较, 不能用以下形式:
 - **`if(str1>str2) printf("yes");`**
 - 而只能用
 - **`if(strcmp(str1, str2)>0) printf("yes");`**

• 6. strlen函数

- 其一般形式为: **strlen (字符数组)**

- strlen是测试字符串长度的函数。函数的值为字符串中的实际长度(不包括' \0' 在内)。

- **例如:** `char str [10] = { " China" };`

- `printf(" %d" , strlen(str));`

- 输出结果不是10, 也不是6, 而是5。也可以直接测试字符串常量的长度, 如`strlen(" China" );`

• 7. **strlwr**函数

- 其一般形式为: **strlwr** (字符串)

- **strlwr**函数的作用是将字符串中大写字母换成小写字母。

• 8. **strupr**函数

- 其一般形式为: **strupr** (字符串)

- **strupr**函数的作用是将字符串中小写字母换成大写字母。

字符数组的特点总结：

- 1、字符数组用来存放字符串，以 ‘\0’ 作为字符串的结束字符。
- 2、由于 ‘\0’，字符数组实际能存放的其他字符个数比数组长度少1；且在处理字符数组时，既可以用数组的长度也可以用 ‘\0’ 来判断字符串是否结束。
- 3、对字符数组输入输出时，除了用循环语句逐个地对数组元素进行输入输出之外，还可用scanf/printf函数、gets/puts函数进行一次性整体输入输出；
- 4、初始化（输入）字符串时，如果是逐个字符的赋值，使用循环结构逐个对字符数组元素赋值时，当赋值结束时，程序员必须将 ‘\0’ 赋值给字符数组。

7.4 字符数组

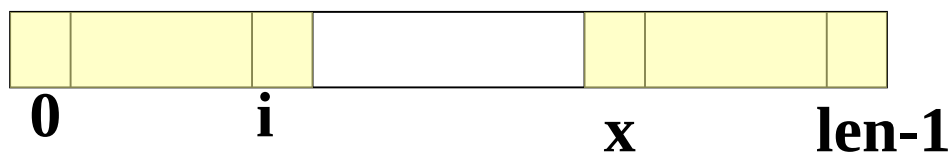
练习2：从键盘读取 一个长度不超过50的字符串（以 ‘#’ 结束）存放到一个字符数组中，然后将字符串逆序，并输出。如输入为：“**hello world#**”，则输出为：**dlrow olleh.**

问题分析：

实现逆序的手段是第*i*个元素和倒数第*i*个元素 $a[x]$ 交换。

交换次数： $len/2$ 次（ len 为字符串长度），故 i 的变化范围是： $0 \sim len/2-1$

$a[i]$ 与 $a[x]$ 交换，则由 $i-0=len-1-x$ ，得 $x=len-1-i$



```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    char string[50],ch;
```

```
    int pos, i, len;
```

```
    //从键盘读取字符串到字符数组中，直到结束符'#'，
```

```
    pos=0;
```

```
    scanf("%c",&ch);
```

```
    while(ch!='#'){
```

```
        string[pos]=ch;
```

```
        pos++;
```

```
        scanf("%c",&ch);
```

```
    }
```

```
    string[pos]= '\0' ;//字符串结束符
```

//输出字符数组

```
printf("the original string is:%s\n",string);
```

//将字符数组中的元素逆序，并输出

```
len = pos;//字符数组中有效字符的个数，不包括 ‘\0’
```

```
for(i=0;i<=len/2-1;i++){ //交换 string[i]和 string[len-1-i]
```

```
    ch=string[i];
```

```
    string[i]=string[len-1-i];
```

```
    string[len-1-i]=ch;
```

```
}
```

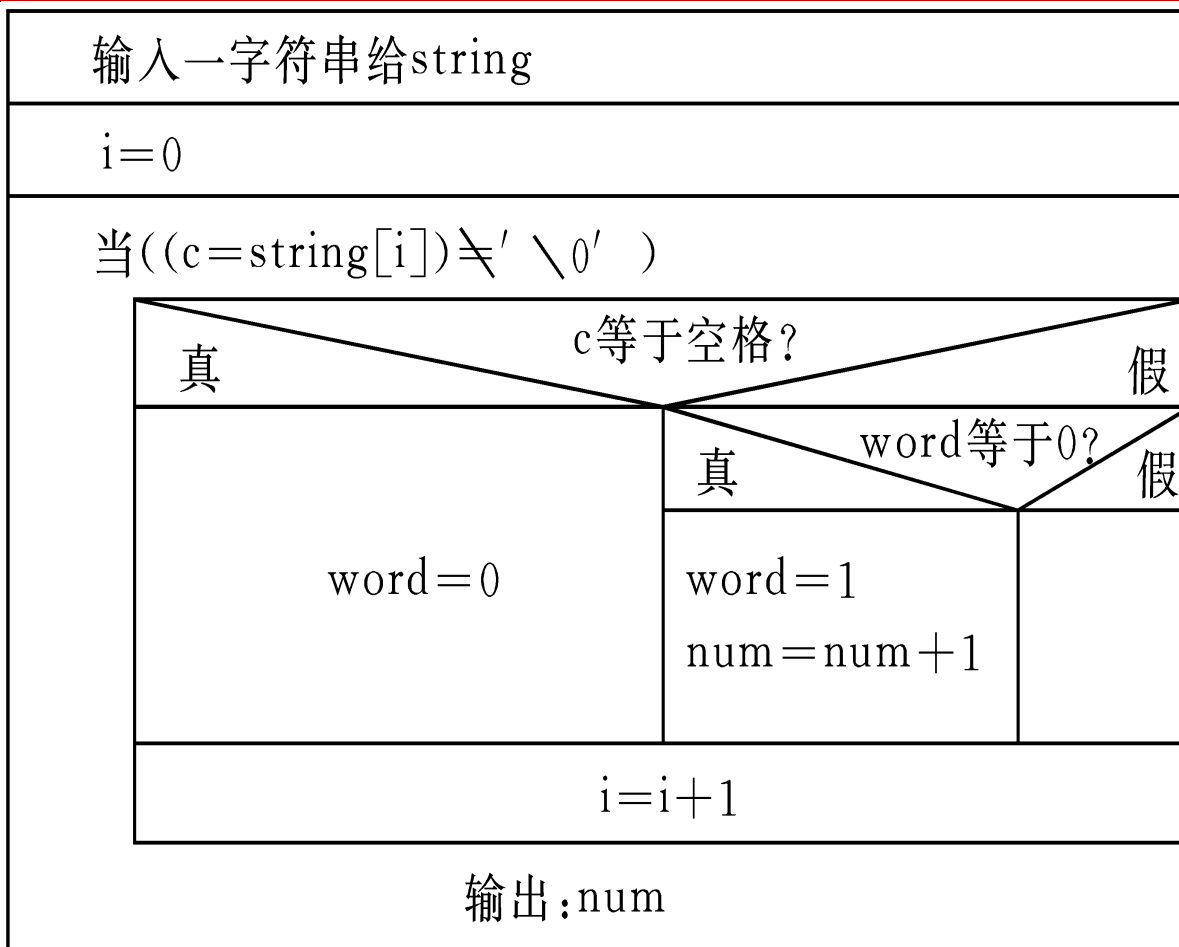
//输出逆序后的字符数组

```
printf("the string after converse is:%s\n",string);
```


//输出逆序后的字符数组 :逐个字符输出

```
printf("the string after converse is:");  
pos = 0;  
while (string[pos]!='\0'){  
    printf(" %c",string[pos]);  
    pos++;  
}  
  
return 0;  
}
```

- 例8 输入一行字符，统计其中有多少个单词，单词之间用空格分隔开。





• 程序如下:

- #include <stdio.h>
- int main()
- {
- char string [81] ;
- int i, num=0, word=0;
- char c;
- gets(string);
- for (i=0;(c=string [i])!=' \ 0';i++)



```
• if(c==' ') word=0;
• else if(word==0)
• { word=1;
•   num++;
• }
• printf("There are %d words in the
•   line. \n", num);
• return 0 ;
• }
```

• 运行情况如下:

• I am a boy. ✓

• There are 4 words in the line.

•例9 有3个字符串, 要求找出其中最大者

•程序如下:

```
•#include<stdio.h>
•#include<string.h>
•int main ( )
•{
•    char string [20] ;
•    char str [3] [20] ;    •二维数组: 等价于定义了
                           3个一维数组
•    int i;
•    for (i=0;i<3;i++)
•        gets (str [i] );
```



- if (strcmp(str [0] ,str [1])>0)
- strcpy(string,str [0])
- else strcpy(string,str [1]);
- if (strcmp(str [2] ,string)>0)
- strcpy(string,str [2]);
- printf(" \nthe largest string is :
- \n%s \n",string);
- return 0 ;
- }



• 运行结果如下:

• CHINA ✓

• HOLLAND ✓

• AMERICA ✓

•

• the largest string is :

• HOLLAND

7.1 总结与回顾（数据类型与程序设计）

7.2 数组--数据的存储结构

7.3 数组的声明、操作和使用

7.4 字符数组（字符串及操作）

7.5 数组作为函数参数的处理

7.6 数组的排序、查找算法



7.5 数组作为函数参数的处理

数组用作函数参数有两种形式，

- 1、将**数组元素**(下标变量)作为实参传递给函数；
- 2、将**数组名**作为实参传递给函数，目的是让被调用函数能访问、操作该数组。

数组名实际上是数组的首元素地址

```
int main(void)
{
    char array[5];
    printf("array = %p &array[0] = %p", array,&array[0])
    return 0;
}
```

```
array = FFF0
&array[0] = FFF0
```

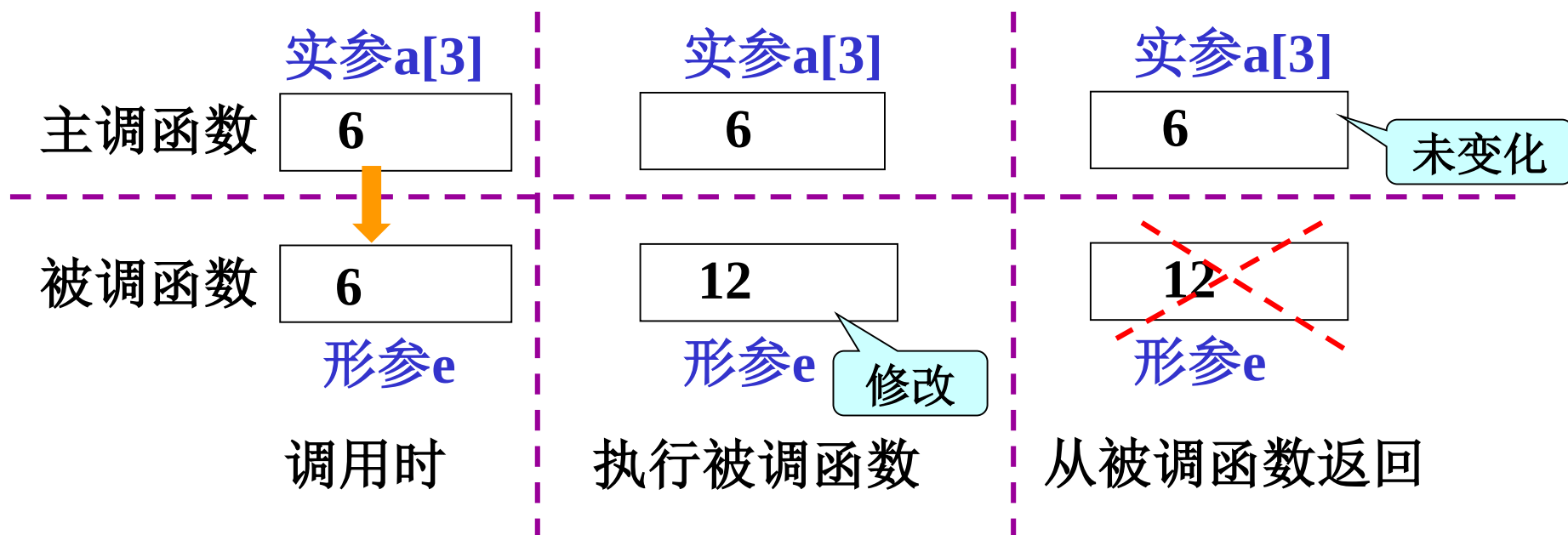
7.5 数组作为函数参数的处理

一、数组元素作函数实参

数组元素就是下标变量，它与普通变量并无区别。因此它作为函数实参使用时与普通变量完全相同，在发生函数调用时，把作为实参的数组元素的值传送给形参，实现单向的按值传送。

```
void modifyElement(int e)
{
    printf("%d",e*=2);
}
```

函数调用: `y= modifyElement (a[3]);`



数组元素作函数实参

7.5 数组作为函数参数的处理

二、将**数组名**作为函数的实参

目的：数组名作为函数的实参时，传递的是数组的首地址（第一个元素的地址）。**使得被调用函数能够访问、操作原数组！**

方法：要求形参和相对应的实参都必须是类型相同的数组，都必须有明确的数组说明，此时被调函数实际操作的是原数组。同时，通常要将数组的大小传递给函数。

机理：将数组名作为实参传递给函数，函数就获得了数组的首地址，根据首地址能计算出原数组各个元素的内存地址，从而访问这些数组元素。

• 地址—引用—指针

• 传地址=传引用

```
void modifyArray(int b[],int size)
```

```
{
```

```
    int j;
```

```
    for (j=0;j<=size-1;j++)
```

```
        b[j]*=2;
```

```
}
```

函数调用: `y= modify(a, SIZE);`

如果函数要接收一个数组进行处理，则形参中必须有一个是同类型的数组，如`int b[]`，用于接收调用函数中数组的首地址。

- `b[j]`的内存地址是: $b+j*\text{sizeof}(\text{int})$
- `a[j]`的内存地址是: $a+j*\text{sizeof}(\text{int})$
- 而`a`和`b`的值相等，所以`a[j]`和`b[j]`是同一个元素

实参a (数组名)

实参a

实参a

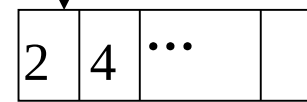
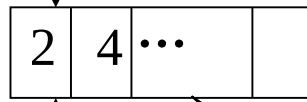
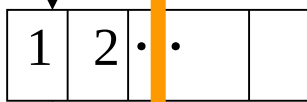
主调函数

a首地址

a首地址

a首地址

数组a



已改变

被调函数

a首地址

a首地址

~~a首地址~~

形参b

形参b

~~形参b~~

调用时

执行被调函数

从被调函数返回

7.5 数组作为函数参数的处理

```
#include<stdio.h>
```

```
#define SIZE 5
```

```
void modifyArray(int [],int);/*函数原型中数组类型的参数，[]中  
不必包含数组的长度*/
```

```
void modifyElement(int);
```

```
int main()
```

```
{
```

```
    int    a[SIZE] = { 0 , 1 , 2 , 3 , 4 } ;
```

```
    int    i;
```

```
    printf("Effects of passing entire array call by reference:\n The  
value of the original array are :\n");
```

7.5 数组作为函数参数的处理

```
for( i = 0 ; i < SIZE; i++ )  
    printf( "%3d" , a[i] );
```

```
modifyArray(a,SIZE);/*数组名作为实参,以传引用方式传递数组a*/
```

```
printf("\nThe value of the modified array are :\n");
```

```
for( i = 0; i < SIZE ; i++ )
```

```
    printf( "%3d" , a[i] );
```

```
printf("\n\nEffects of passing array element call by  
value:\n\n")
```

```
printf ("the value of a[3] is %d \n",a[3]);
```

```
modifyElement(a[3]);
```

```
printf("The value of a[3] is %d",a[3]);
```

```
return 0;
```

```
}
```

7.5 数组作为函数参数的处理

/*数组作为函数形参，[]中不必包含数组的长度，如果包括了，编译器会将其忽略掉； 将数组传递给函数时，通常要将数组的大小传递给函数*/

```
void modifyArray(int b[],int size)
```

```
{
```

```
    int i;
```

```
    for( i = 0 ; i < size ; i++ )
```

```
        b[i] *= 2 ;
```

```
}
```

```
void modifyElement(int e)
```

```
{
```

```
    printf("Value in modify element is %d\n",e*=2);
```

```
}
```


7.5 数组作为函数参数的处理

函数定义: **void modifyArray(int b[], int size)**

/*数组作为函数形参，[]中不必包含数组的长度，如果包括了，编译器会将其忽略掉；将数组传递给函数时，通常要将数组的大小传递给函数*/

函数原型: **void modifyArray(int [], int);**

/*函数原型中数组类型的参数，[]中不必包含数组的长度*/

函数调用: **modifyArray(a, SIZE);**

/*数组名称作为实参,以传引用方式传递数组a*/

7.5 数组作为函数参数的处理

- 由于数组总是通过**模拟传引用**的方式传递的，所以难以防范函数修改数组的值。为了限制函数对数组的修改，可以使用**const**限定符。

```
void tryToModifyArray(const int b[],int size)
{
    int i;
    for(i=0;i<size;i++)
        b[i]=2*b[i]; //编译报错， b[i]不能修改
}
```

7.5 数组作为函数参数的处理

练习：设计一个函数**outputArray**，用于输出一个整数数组中的元素

```
void outputArray(const int data[], int size)
```

```
{  
    int i;  
    for(i = 0; i < size; i++)  
        printf(" %d ", data[i]);  
    printf("\n");  
}
```

用**const**做数组参数的前缀，使得在 **outputArray**函数中不允许对数组b的元素进行修改。

7.5 数组作为函数参数的处理

```
#define SIZE 10
void outputArray(const int a[],int size);
int main(void)
{
    int a[SIZE]={2,1,6,9,8,36,27,25,12,7};
    outputArray (a,SIZE); //输出a中的所有元素
    outputArray (&a[1],SIZE-1); /*输出a中第2个元素开始
                                   的所有元素*/

    return 0;
}
```

数组元素求和-递归算法1

- 设计一个递归函数，用于求整型数组中各元素之和。

//递归求数组元素之和—算法1

```
int sum1(int a[], int size)
{
    if (size == 1)
        return a[0];
    else
        return sum1(a,size-1) + a[size-1];
}
```

数组元素求和-递归算法2

//递归求数组元素之和—算法2

```
int sum2(int a[],int size)
{
    if (size == 1)
        return a[0];
    else
        return a[0] + sum2(&a[1],size -1 );
}
```

```
#define SIZE 5
int main()
{
    int a[SIZE ]={1,2,3,4,5};

    printf("sum1=%d\n",sum1(a, SIZE ));
    printf("sum1=%d\n",sum2(a, SIZE ));

    return 0;
}
```

7.5 数组作为函数参数的处理

- 请说明下述程序的功能。 [源程序](#)

```
void someFunction(int b[], int size)
{
    if(size > 0){
        someFunction(&b[1], size-1);
        printf("%4d", b[0]);
    }
}
```

注意：数组不能作为函数的返回值；

7.1 总结与回顾（数据类型与程序设计）

7.2 数组--数据的存储结构

7.3 数组的声明、操作和使用

7.4 字符数组（字符串及操作）

7.5 数组作为函数参数的处理

7.6 数组的排序、查找算法



7.6 数组的排序与查找算法

排序：

现实世界中排序无处不在，排序也是算法方法中的经典问题。假定待排序的 n 个数据，原本是杂乱无章的存放在数组中，我们需要设计算法将这些数排序。

- 冒泡排序
- 选择排序
- 插入排序
- . . .

查找：

同样，查找也是现实世界中经常遇到的问题，如：字典查找、搜索引擎等。

- 顺序查找
- 折半查找

7.6 数组的排序与查找算法

数组排序（冒泡排序）

（参见书图6-14）

升序排序算法设计思路[\[链接\]](#)：

- 1、每一趟过程通过两两比较和交换将最大的元素冒泡到最后(下标为0的元素最靠前)；
- 2、需要 $(N-1)$ 趟过程完成数组排序（ N 是元素个数）；

若是升序排序：每一趟过程都是通过两两比较和交换，将最大的元素“冒泡”到最后。

初始数组：

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
4	5	7	1	2	3

第一趟排序， 确定a[5]	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
	<u>4</u>	<u>5</u>	7	1	2	3
4和5比较，不用交换	4	<u>5</u>	<u>7</u>	1	2	3
5和7比较，不用交换	4	5	<u>7</u>	1	2	3
7和1比较，需要交换	4	5	1	<u>7</u>	2	3
7和2比较，需要交换	4	5	1	2	<u>7</u>	3
7和3比较，需要交换	4	5	1	2	3	<u>7</u>

7.6 数组的排序与查找算法

初始数组: ↵

$a[0]$ ↵	$a[1]$ ↵	$a[2]$ ↵	$a[3]$ ↵	$a[4]$ ↵	$a[5]$ ↵	↵
4 ↵	5 ↵	7 ↵	1 ↵	2 ↵	3 ↵	↵

第一趟排序: 确定 $a[5]$

$a[0]$ ↵	$a[1]$ ↵	$a[2]$ ↵	$a[3]$ ↵	$a[4]$ ↵	$a[5]$ ↵	↵
4 ↵	5 ↵	1 ↵	2 ↵	3 ↵	7 ↵	↵

$a[2]$ 和 $a[3]$ 交换, $a[3]$ 和 $a[4]$ 交换, $a[4]$ 和 $a[5]$ 交换 ↵

第二趟排序: 确定 $a[4]$

$a[0]$ ↵	$a[1]$ ↵	$a[2]$ ↵	$a[3]$ ↵	$a[4]$ ↵	$a[5]$ ↵	↵
4 ↵	1 ↵	2 ↵	3 ↵	5 ↵	7 ↵	↵

$a[1]$ 和 $a[2]$ 交换, $a[2]$ 和 $a[3]$ 交换, $a[3]$ 和 $a[4]$ 交换 ↵

7.6 数组的排序与查找算法

第三趟排序：确定a[3]



a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
1	2	3	4	5	7

a[0]和 a[1]交换， a[1]和 a[2]交换， a[2]和 a[3]交换

第四趟排序：确定a[2]



a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
1	2	3	4	5	7

第五趟排序：确定a[1]

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
1	2	3	4	5	7



【返回】

7.6 数组的排序与查找算法

```
for(loc = size-1; loc >= 1; loc--)
```

确定 $a[loc]$ 的值



```
for(i = 0; i <= loc-1; i++)
```

若 $a[i] > a[i+1]$, 则交换

确定 $a[loc]$: 需要 $a[0] \sim a[loc]$ 这段范围的元素两两比较交换:

if ($a[i] > a[i+1]$) 则 $a[i]$ 与 $a[i+1]$ 交换

i 的变化范围: $0 \sim loc-1$

思考: 如果要想实现降序排序, 上述算法如何修改?

若 $a[i] < a[i+1]$, 则交换

7.6 数组的排序与查找算法

/*冒泡排序算法1:升序排序,以loc作为循环控制变量*/

void bubbleSort1(int a[],int size)

{

int loc,i,temp;

/*总共需要比较size-1趟。每一趟确定a[loc]的值*/

for(loc = size-1;loc >= 1;loc--){

/*从下标为0~loc数组元素中依次进行比较交换*/

for(i = 0;i <= loc-1;i++)

if(a[i] > a[i+1]){ /*相邻两个元素交换*/

temp = a[i];

a[i] = a[i+1];

a[i+1] = temp;

}

}

}

/*冒泡排序算法2:升序排序,以pass作为循环控制变量*/

void bubbleSort2(int a[],int size)

{

int pass,i,temp;

/*总共需要比较size-1趟。每一趟确定a[size-pass]的值*/

for(pass=1;pass<=size-1;pass++){

/*从下标为0~size-pass数组元素中依次进行比较交换*/

for(i=0;i<=size-pass-1;i++){

if(a[i]>a[i+1]){ /*交换*/

temp=a[i];

a[i]=a[i+1];

a[i+1]=temp;

}

}

}

7.6 数组的排序与查找算法

课后练习：

设计冒泡排序算法，实现降序排序。要求：每一趟过程通过两两比较和交换将最大的元素往前冒泡（下标为0的元素最靠前）。

例如，第一趟，是从 $a[\text{size}-1]$ 元素开始，每个元素和其前面一个元素两两比较，若大于前面元素，则交换。

7.6 数组的排序与查找算法

```
void bubbleSort3(int a[],int size)
{
    int loc,i,temp;

    /*总共需要比较size-1趟。每一趟确定a[loc]的值*/
    for(loc = 0;loc <= size-2;loc++){
        /*从下标为size-1~loc数组元素中依次进行比较交换*/
        for(i = size-1;i >= loc+1;i--){
            if(a[i] > a[i-1]){ /*相邻两个元素交换*/
                temp = a[i];
                a[i] = a[i-1];
                a[i-1] = temp;
            }
        }
    }
}
```

7.6 数组的排序与查找算法

数组排序（选择排序）

降序排序算法设计思路[\[链接\]](#):

从 $a[0] \sim a[\text{size}-1]$ 这段元素中找最大元素 $a[\text{max}]$ ， $a[0]$ 和 $a[\text{max}]$ 交换；接着，从 $a[1] \sim a[\text{size}-1]$ 这段元素中找最大元素 $a[\text{max}]$ ， $a[1]$ 和 $a[\text{max}]$ 交换；依次类推，直到第 $N-1$ 个元素为止。

程序设计考虑：

利用查找数组最大元素的函数对线性表降序排序

[算法设计](#)

7.6 数组的排序与查找算法

初始数组：

$a[0]$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$
4	5	7	1	2	3

第一趟排序： $a[2]$ 为0~5中最大值， $a[0]$ 和 $a[2]$ 交换

$a[0]$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$
7	5	4	1	2	3

第二趟排序： $a[1]$ 为1~5中最大值，不交换

$a[0]$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$
7	5	4	1	2	3

第三趟排序：a[2] 为 2~5 中最大值，不交换

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
7	5	4	1	2	3



第四趟排序：a[5] 为 3~5 中最大值，a[3]和 a[5]交换

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
7	5	4	3	2	1



第五趟排序：a[4] 为 4~5 中最大值，不交换

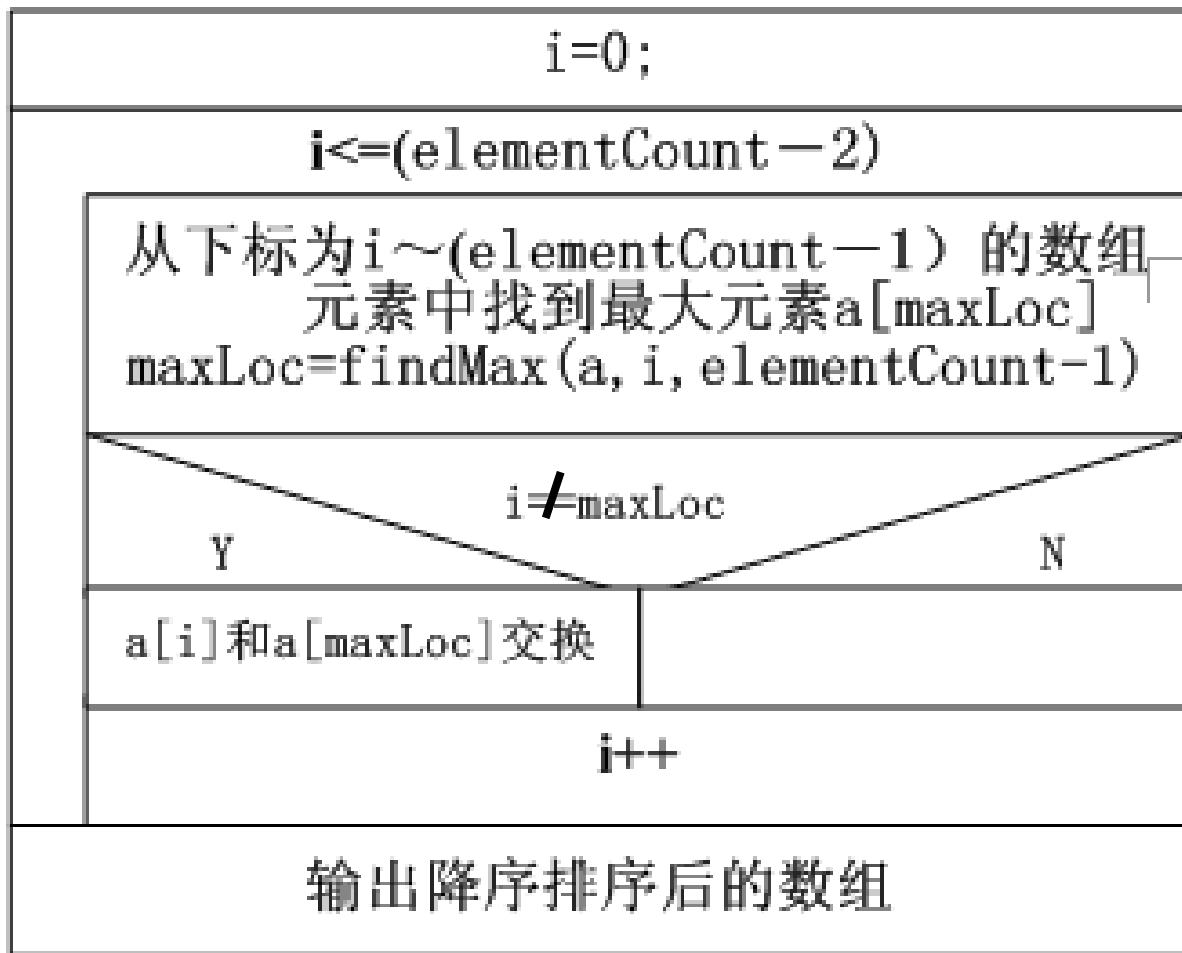
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
7	5	4	3	2	1



【返回】

每循环一次，确定 $a[i]$ 的值, i 取值范围：
 $0 \sim \text{elementCount}-2$

选择排序—降序：



/* 函数功能：选择排序（降序）

参数说明：数组，数组中已有元素个数*/

```
void selectSortDown(int data[],int elementCount)
{
    int i,maxLoc,temp;

    for (i = 0; i <= elementCount-2; i++){
        maxLoc = findMax(data,i,elementCount - 1);
        if (i != maxLoc){
            temp = data[i];
            data[i] = data[maxLoc];
            data[maxLoc] = temp;
        }
    }
}
```


7.6 数组的排序与查找算法

查找表中最大元素的下标

函数设计考虑：

参数设计：

- 数组作为参数传入；
- 查找起始位置作为参数传入；
- 查找结束位置作为参数传入；

返回结果：最大元素的位置（下标）

功能设计：顺序对数组的元素逐个处理（比较）。

7.6 数组的排序与查找算法

/* 函数功能：找数组中的最大值元素，并返回其下标

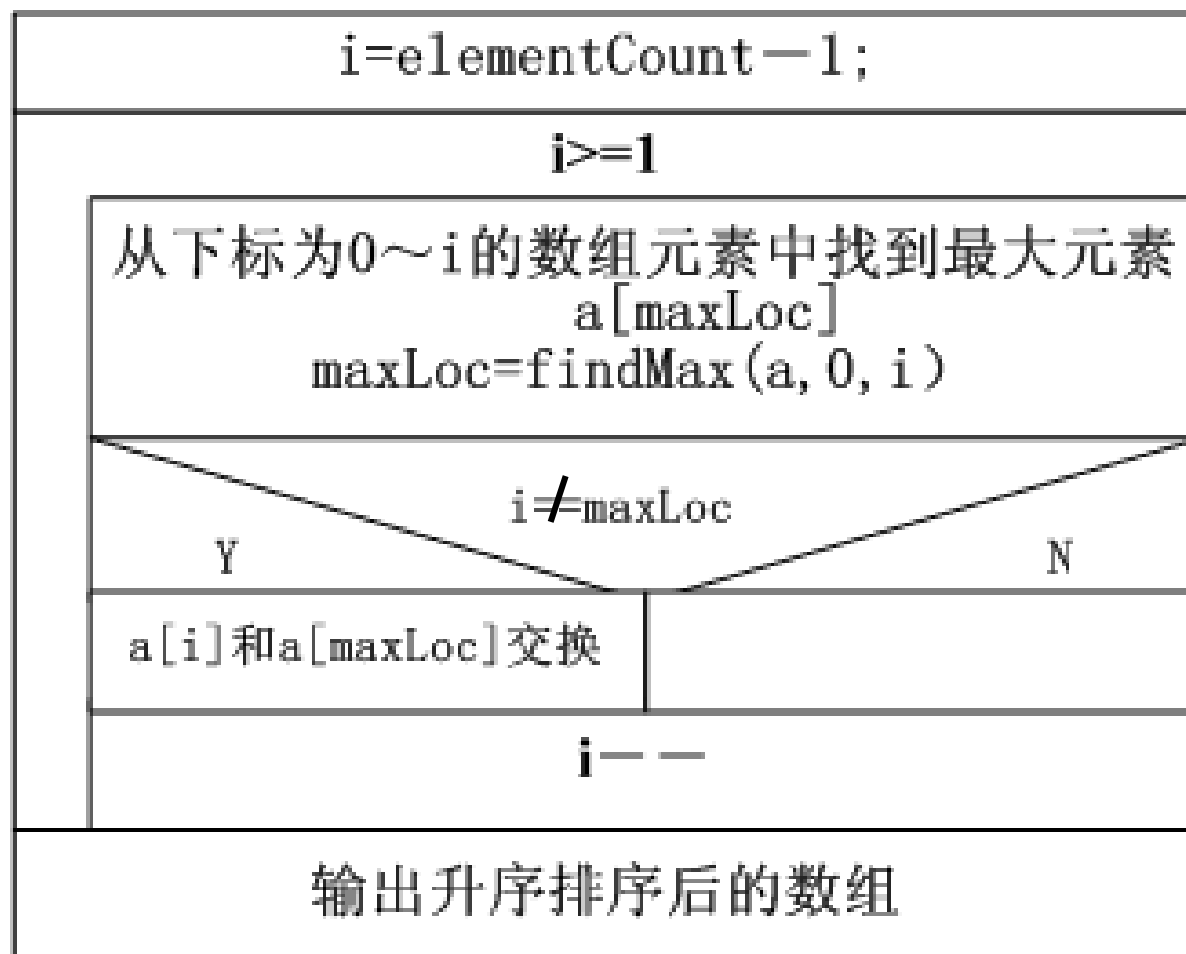
参数说明：数组名，查找起始位置下标，查找终止位置下标*/

```
int findMax(int data[], int startLoc, int endLoc)
{
    int i, maxloc;

    maxloc = startLoc;
    for (i = startLoc+1; i <= endLoc; i++)
        if (data[i] > data[maxloc])
            maxloc = i;

    return maxloc;
}
```

选择排序—升序:



7.6 数组的排序与查找算法

查找数组元素

设计一个函数，查找某个元素在数组中的下标并返回。

若存在多个符合条件的元素，则只返回第一个符合条件元素的下标。

函数设计考虑：

参数设计：

- 数组名为参数传入
- 查找起始位置作为参数传入；
- 查找结束位置作为参数传入；
- 要查找的值作为参数传入；

返回结果： 若找到，则返回元素下标，否则返回-1；

7.6 数组的排序与查找算法

/* 函数功能：在数组的data[startLoc]~data[endLoc]这段数组元素中
查找某个元素，如果找到则返回该元素的下标，否则返回-1。*/

int findElement(const int data[],int startLoc, int endLoc, int element)

```
{  
    int i;  
    i = startLoc;  
    while ( i <= endLoc && data[i] != element)  
        i++;  
  
    if ( i <= endLoc)/*若找到，思考：能否写成if(data[i]==element)? */  
        return i;  
    else  
        return -1;  
}
```

i = 下标初始值

i未越界且a[i]不符合查找要求

i++

7.6 数组的排序与查找算法

线性查找（假设数组元素是无序排列的）

从第一个元素开始依次找，直到找到或全部找完但没有找到。

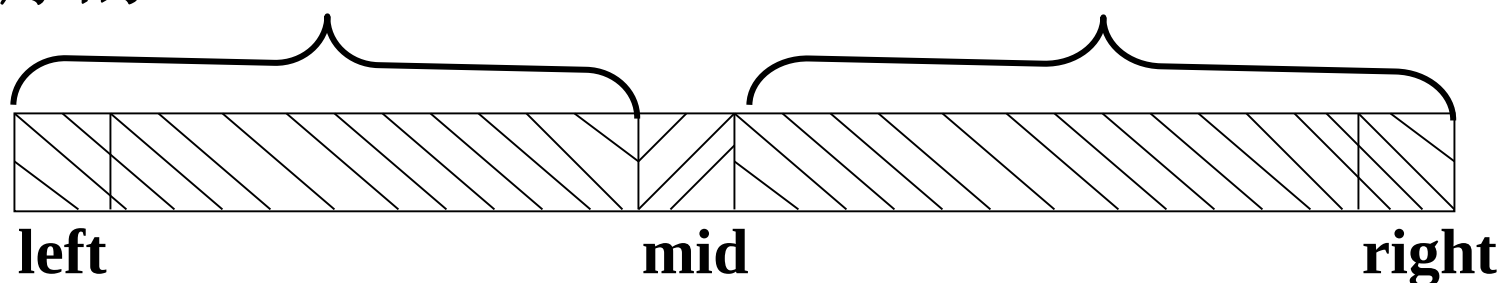
```
int linearSearch(int array[],int key,int size)
{
    int n,found;
    for (n = 0, found = -1; n <= size-1 && found == -1; n++)
        if (array[n] == key)
            found = n;
    return found;
}
```

7.6 数组的排序与查找算法

```
int linearSearch(int array[],int key,int size)  
{  
    int n;  
  
    for (n = 0; n <= size-1; n++)  
        if (array[n] == key)  
            return n;  
  
    return -1;  
}
```

7.6 数组的排序与查找算法

折半查找（要求数组元素是已排序的，假设是升序排序的）

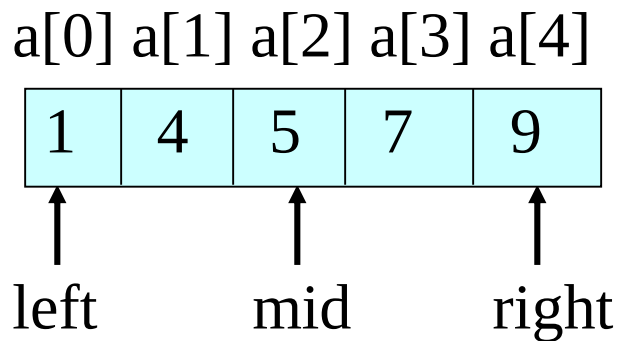


在下标为 $\text{left} \sim \text{right}$ 的元素中查找值为 key 的元素下标：
先求出 $\text{mid} = (\text{left} + \text{right}) / 2$ ；若 $\text{key} == \text{a}[\text{mid}]$ ，说明找到；
若 $\text{key} < \text{a}[\text{mid}]$ ，说明 key 可能在下标为 $\text{left} \sim \text{mid} - 1$ 的这段元素中，修正查找范围，继续查找。
若 $\text{key} > \text{a}[\text{mid}]$ ，说明 key 可能在下标为 $\text{mid} + 1 \sim \text{right}$ 的这段元素中，修正查找范围，继续查找。通过大范围缩减查找范围来提高有序表的查找速度。

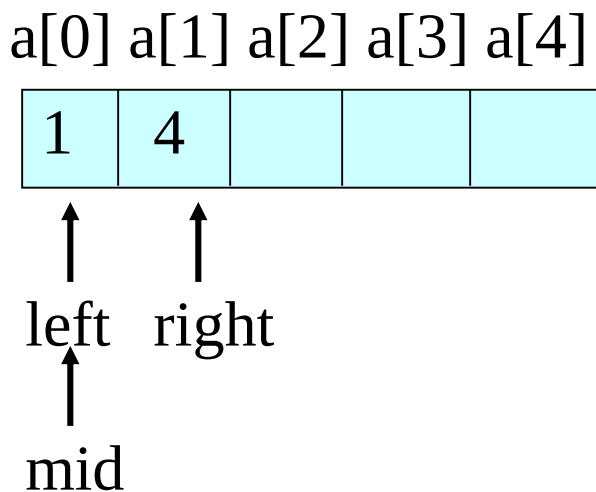


在下标为left~right的元素中查找值为key的元素下标。例如：查找3。 $mid = (left + right) / 2$;

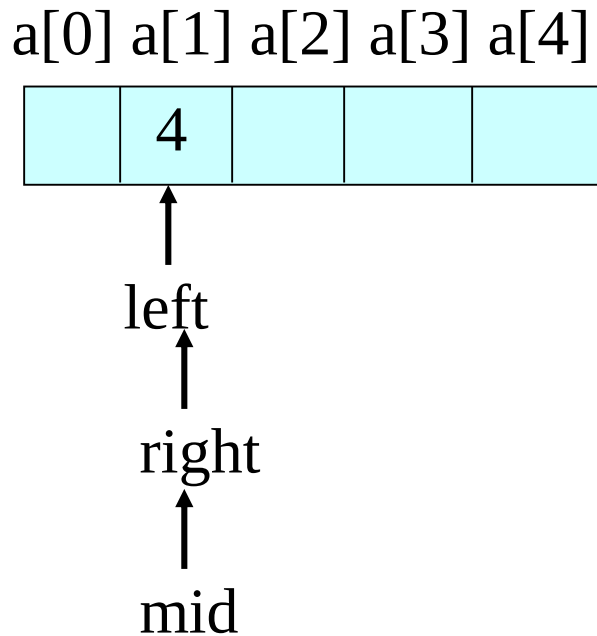
第一趟



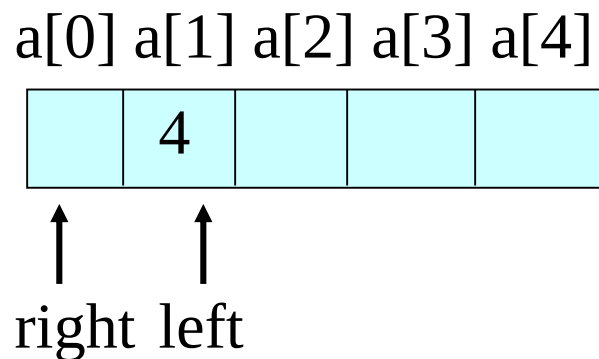
第二趟



第三趟



第四趟



循环结束条件：left>right 或者已经找到

```
int binarySearch(int a[],int searchKey,int low,int high)
{
    int middle;
    while(low <= high){
        middle = (low + high)/2;
        if (searchKey == a[middle])
            return middle;
        else if(searchKey < a[middle])
            high = middle - 1;
        else
            low = middle + 1;
    }
    return -1;
}
```

```
int binarySearch(int a[],int searchKey,int low,int high)
{
    int middle;
    middle = (low + high)/2;
    while(low <= high && a[middle]!=searchKey){
        if(searchKey < a[middle])
            high = middle - 1;
        else
            low = middle + 1;
        middle = (low + high)/2;
    }
    if (low > high) return -1;
    else return middle;
}
```